

Relatório de Auditoria de Segurança

ddlab-data

09 de setembro de 2026

Escopo auditado

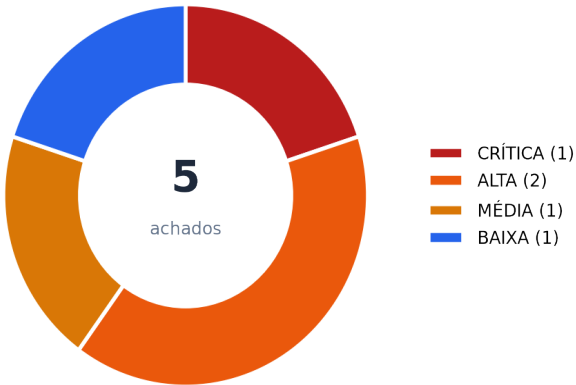
Repositório **dfassuncao/ddlab-data** (branch de trabalho `claude/ddlab-security-analysis-ter837`): aplicação interna da agência DDLab para inteligência de campanhas Google Ads/GA4 — Cloudflare Worker (Hono) como backend/API, SPA React como frontend, Cloudflare D1 (SQLite) como banco pré-agregado, BigQuery como fonte de dados e Cloudflare Access como camada de login. Todos os arquivos de `worker/`, `src/`, `shared/`, `migrations/` e arquivos de configuração/deploy (`wrangler.toml`, `DEPLOY.md`, `.dev.vars.example`, `package.json`) foram revisados; o histórico completo do git foi varrido em busca de segredos commitados.

Nota metodológica — mapeamento das 5 categorias para esta stack

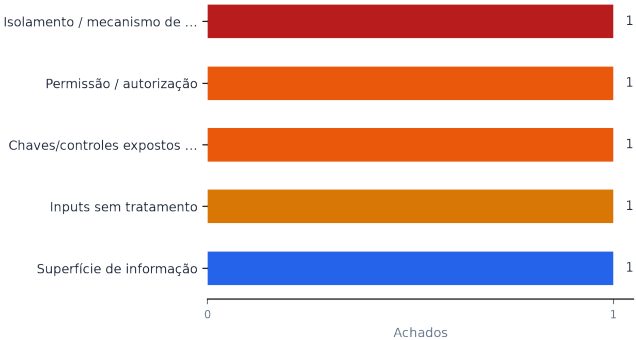
- 1. Isolamento de inquilino/dono:** a aplicação é single-tenant por desenho (uma agência, múltiplas contas de clientes finais, sem conceito de usuário-por-cliente) — o mecanismo de "tranca" é o Cloudflare Access na borda mais a verificação de JWT em `worker/auth.ts`. Auditado como "esse mecanismo pode falhar aberto?" e "há segmentação de dados por conta dentro do banco?".
- 2. Permissão definida no navegador:** mapeado como "o frontend esconde alguma ação atrás de `papel/isAdmin` e o backend replica essa checagem?" — buscado em todo `src/` por lógica de `papel/permissoão`.
- 3. IDOR:** todos os handlers de rota de `worker/routes/*.ts` e `worker/index.ts` foram percorridos individualmente, verificando se buscas/updates por `?account=` ou `id` no corpo confirmam posse.
- 4. Chaves expostas:** checado `wrangler.toml`, `.dev.vars.example`, `package.json`, `migrations`, `seed/`, `DEPLOY.md/README.md` e histórico do git (`git log --all -p`) por segredo real, e cada var não-secreta com default vazio por ausência de validação de startup.
- 5. Inputs sem tratamento (XSS):** mapeado para React (`dangerouslySetInnerHTML`, renderização de Markdown) no frontend, e para escaping em exportação CSV/templates no backend.

Resumo executivo

Severidade	Qtde
CRÍTICA	1
ALTA	2
MÉDIA	1
BAIXA	1



Achados por severidade



Achados por categoria

A auditoria percorreu 100% dos handlers de rota do backend (worker/index.ts e worker/routes/*.ts), o middleware de autenticação, a camada de acesso a dados (D1 e BigQuery), o ETL e os pontos de renderização do frontend. Foram identificados **5 achados** — destaque para **F1 (crítico)**: a autenticação do Cloudflare Access falha aberta quando as variáveis de configuração não estão presentes, e esse é justamente o valor commitado por padrão no repositório. Como pontos fortes, o código evita consistentemente SQL injection (D1 e BigQuery) e XSS via HTML bruto, e nenhum segredo real foi encontrado no histórico do git.

Pontos fortes

O que já está protegido, com a evidência verificada no código.

PONTO FORTE

Todas as queries ao D1 usam parâmetros bind (?), sem concatenação de entrada do usuário em SQL

Verificado em `worker/routes/reports.ts`, `analysis.ts`, `decision.ts` e `rules.ts` — o helper ``q(env, sql, binds)`` sempre usa ``bind(...binds)``. Os únicos identificadores de tabela/coluna interpolados diretamente no SQL vêm de mapas estáticos no código (``KINDS`` em `reports.ts`, ``FACTS`` em `etl/queries.ts`), nunca de query string ou body do usuário.

PONTO FORTE

Consultas ao BigQuery neutralizam identificadores controláveis pelo usuário antes de interpolar no SQL

`worker/etl/ga4.ts:5,90-92` valida ``ga4_dataset`` e cada item de ``ga4_key_events`` (campos editáveis em Configurações) com o regex ``^[a-zA-Z0-9_]+$`` (safelident) antes de usá-los na montagem da query GA4; ``customer_id`` é passado como parâmetro nomeado ``@customer_id`` do BigQuery (`worker/etl/run.ts:60-63`), não concatenado.

PONTO FORTE

Segredos reais nunca foram commitados no repositório

`.gitignore` exclui `.dev.vars`, `service-account*.json`, `*.key.json` e o padrão específico do arquivo baixado do GCP (`studio-*.json`). ``git log --all -p`` não retornou nenhuma chave privada, API key ou token real — apenas o placeholder `"..."` em `.dev.vars.example`. Segredos (`GCP_SA_KEY`, `ANTHROPIC_API_KEY`, `CF_ACCESS_*`) são aplicados via ``wrangler secret put``, fora do controle de versão.

PONTO FORTE

Verificação de JWT do Cloudflare Access implementada corretamente quando configurada

`worker/auth.ts:58-86` valida assinatura RS256 contra o JWKS público do Access (cache de 1h), expiração (``exp``) e audience (``aud``, aceitando array ou string) antes de aceitar o token — nenhuma dessas checagens é pulada quando as vars estão presentes.

PONTO FORTE

Nenhum XSS de HTML bruto no frontend: conteúdo Markdown gerado por IA é renderizado com sanitização por padrão

`src/pages/AiAnalysis.tsx` usa `` sem o plugin ``rehype-raw`` — por padrão o `react-markdown` NÃO renderiza tags HTML embutidas no texto, apenas Markdown. Busca em todo ``src/`` por ``dangerouslySetInnerHTML`/`innerHTML`` não encontrou nenhum outro ponto de renderização de HTML não confiável.

PONTO FORTE

Geração de JWT para a service account do GCP usa Web Crypto nativa, sem expor a chave privada em logs

`worker/bq.ts:30-78` assina o JWT com ``crypto.subtle.sign`` e cacheia apenas o access token (não a chave privada) em memória do isolate, com expiração controlada.

Pontos fracos (riscos centrais)

O risco central é estrutural: o mecanismo único de "tranca" (Cloudflare Access) pode falhar aberto por configuração ausente (F1), e, mesmo quando ligado, não há segunda camada de autorização — todo usuário autenticado é, na prática, administrador de todas as contas de todos os clientes (F2), incluindo a capacidade de gerar custo real em serviços pagos de terceiros sem controle adicional (F3).

Achados detalhados

Isolamento / mecanismo de tranca

Sev.	Arquivo:linha	Descrição
CRÍTICA	worker/auth.ts:88-94 (+ wrangler.toml:29-31)	Autenticação falha aberta (fail-open) quando CF_ACCESS_TEAM_DOMAIN/CF_ACCESS_AUD não estão configurados accessMiddleware() só valida o JWT do Cloudflare Access se as vars CF_ACCESS_TEAM_DOMAIN e CF_ACCESS_AUD estiverem preenchidas. Se qualquer uma estiver vazia — que é exatamente o valor commitado em wrangler.toml (linhas 30-31, CF_ACCESS_TEAM_DOMAIN = "" e CF_ACCESS_AUD = "") — a verificação é pulada inteiramente e o usuário vira { email: "dev@localhost" } com acesso total, sem exigir nenhuma credencial.

Permissão / autorização

Sev.	Arquivo:linha	Descrição
ALTA	worker/routes/reports.ts:438-491 worker/index.ts:41-53 worker/routes/analysis.ts:367-442	Ausência total de RBAC — qualquer usuário autenticado tem acesso irrestrito de leitura e escrita a todas as contas de clientes Não existe conceito de papel (admin/editor/leitor) nem de dono de conta em nenhuma camada do backend. accessMiddleware() só verifica "é um e-mail permitido pela policy do Cloudflare Access" — a partir daí, o mesmo usuário pode ler dados de TODAS as 5 contas (GET /api/report/*, /api/overview, /api/waste, /api/opportunities, /api/decision-center), editar metas/orçamento/briefing de qualquer conta (POST /api/settings/account), criar anotações em nome de qualquer conta (POST /api/annotations), disparar ETL pago para qualquer conta (POST /api/refresh) e gerar análises de IA pagas para qualquer conta (POST /api/analysis, POST /api/diagnostico). Não há checagem de posse — é tudo-ou-nada.

Chaves/controles expostos ou inertes

Sev.	Arquivo:linha	Descrição
ALTA	worker/index.ts:41-53 .dev.vars.example:15-16	Endpoint POST /api/refresh dispara custo real (BigQuery) sem limite de taxa; a proteção REFRESH_TOKEN documentada nunca é verificada no código .dev.vars.example documenta REFRESH_TOKEN como "Token simples para proteger POST /api/refresh quando chamado fora do Access", e Env.REFRESH_TOKEN existe em worker/env.ts:15. Porém nenhum arquivo do worker lê ou compara REFRESH_TOKEN — o handler `app.post("/api/refresh", ...)` em worker/index.ts:41 dispara `runEtl()` (que consulta o BigQuery, custo real por bytes processados) sem checar esse token nem qualquer outro controle além do accessMiddleware() genérico, e sem rate limiting.

Inputs sem tratamento

Sev.	Arquivo:linha	Descrição
MÉDIA	src/pages/ReportPage.tsx:52-68	Exportação CSV de negativas não neutraliza injeção de fórmula (CSV/Excel formula injection) downloadNegativesCsv() escapa aspas duplas nos campos `label` (termo de busca) e `campaign`, mas não verifica se o valor começa com `=`, `+`, `-` ou `@` — caracteres que o Excel/Google Sheets interpretam como início de fórmula. `label` nesse relatório vem de `search_term` do Google Ads (fact_searchterm_daily), que por sua vez reflete termos de busca reais digitados por usuários — um dado de origem externa que a agência não controla totalmente.

Superfície de informação

Sev.	Arquivo:linha	Descrição
BAIXA	worker/index.ts:23-38	GET /api/introspect expõe o schema interno do BigQuery (nomes de coluna/tabela) para qualquer usuário autenticado O endpoint de depuração /api/introspect roda `SELECT column_name, data_type FROM INFORMATION_SCHEMA.COLUMNS` para a tabela informada via querystring e devolve o schema. Está corretamente sob accessMiddleware() e o parâmetro `table` é validado por regex (`^[a-zA-Z0-9_]+\$`) antes de entrar na query (sem SQL injection), mas é uma rota de debug interno deixada acessível em produção para qualquer usuário do Access, não só para quem ajusta o ETL.

Evidência técnica por achado

CRÍTICA

F1 · Autenticação falha aberta (fail-open) quando CF_ACCESS_TEAM_DOMAIN/CF_ACCESS_AUD não estão configurados

worker/auth.ts:88-94 (+ wrangler.toml:29-31)

```
if (!CF_ACCESS_TEAM_DOMAIN || !CF_ACCESS_AUD) {
  c.set("user", { email: "dev@localhost" });
  return next();
}
```

Condições de explorabilidade: Condição de exploração: deploy feito antes do passo 4 do DEPLOY.md (criar a Cloudflare Access Application e rodar `wrangler secret put CF\_ACCESS\_TEAM\_DOMAIN/AUD`), ou qualquer redeploy que rode `wrangler deploy` sem os secrets aplicados (os secrets não são reaplicados automaticamente e o `[vars]` do wrangler.toml sobrescreve com string vazia os nomes homônimos se alguém mover para vars por engano). Nesse cenário a aplicação inteira — dados de custo/receita/conversão de 5 contas de clientes reais, edição de metas e orçamento, criação de anotações e disparo de refresh pago — fica acessível sem login a qualquer pessoa na internet que descubra a URL do Worker.

ALTA

F2 · Ausência total de RBAC — qualquer usuário autenticado tem acesso irrestrito de leitura e escrita a todas as contas de clientes

worker/routes/reports.ts:438-491, worker/index.ts:41-53, worker/routes/analysis.ts:367-442

```
reports.post("/settings/account", async (c) => {
  const body = await c.req.json<{ id: string; ... }>();
  if (!body.id) return c.json({ error: "id obrigatório" }, 400);
  await c.env.DB.prepare(`UPDATE dim_account SET ... WHERE id = ?`)...
  // nenhuma verificação de quem pode editar a conta `body.id`
});
```

Condições de explorabilidade: Sempre explorável por qualquer conta liberada na policy do Cloudflare Access (hoje: dfassuncao@gmail.com e/ou @ddlab.com.br, conforme DEPLOY.md). É um risco aceitável se o produto é deliberadamente "ferramenta interna de agência, todo colaborador vê tudo" — mas isso não está documentado como decisão de produto em lugar nenhum, e cresce de risco assim que a policy do Access for ampliada (ex.: incluir um cliente final, um estagiário, um freelancer pontual).

ALTA

F3 · Endpoint POST /api/refresh dispara custo real (BigQuery) sem limite de taxa; a proteção REFRESH_TOKEN documentada nunca é verificada no código

worker/index.ts:41-53, .dev.vars.example:15-16

```
app.post("/api/refresh", async (c) => {
  const account = c.req.query("account") || undefined;
  // ... nenhuma verificação de c.env.REFRESH_TOKEN
  c.executionCtx.waitUntil(runEtl(c.env, { accountIds: ..., lookbackDays: ..., facts }));
  return c.json({ started: true, ... });
});
```

Condições de explorabilidade: Explorável por qualquer usuário autenticado (ou, combinado com F1, por qualquer visitante não autenticado): chamadas repetidas a /api/refresh?days=400 para as 5 contas geram consultas pesadas e repetidas ao BigQuery (a doc do próprio README/DEPLOY recomenda backfill de 400 dias), inflando a fatura do GCP sem limite.

MÉDIA

F4 · Exportação CSV de negativas não neutraliza injeção de fórmula (CSV/Excel formula injection)

src/pages/ReportPage.tsx:52-68

```
const term = String(r.label ?? "").replace(/"/g, '"');
const camp = String(r.campaign ?? "").replace(/"/g, '"');
return `${camp}`, `${term}`, Phrase, Campaign`;
```

Condições de explorabilidade: Exige que (a) um termo de busca real acionando os anúncios do cliente contenha uma fórmula válida de planilha logo no início do texto — algo incomum mas não impossível de ser induzido por um agente malicioso pesquisando termos elaborados —, e (b) o operador da DDLab abra o CSV exportado no Excel/Sheets com fórmulas habilitadas. Severidade limitada pela baixa probabilidade de ambas condições, mas o padrão de exportação deveria ser seguro por padrão.

BAIXA

F5 · GET /api/introspect expõe o schema interno do BigQuery (nomes de coluna/tabela) para qualquer usuário autenticado

worker/index.ts:23-38

```
app.get("/api/introspect", async (c) => {  
  const table = c.req.query("table");  
  if (!table || !/^[a-zA-Z0-9_]+$/.test(table)) return c.json({ error: "?table inválido" }, 400);  
  ...  
});
```

Condições de explorabilidade: Baixo risco — exige já estar autenticado via Access; vaza apenas nomes/tipos de coluna do dataset do BigQuery, não dados de linha.

Recomendações priorizadas

Prio.	Recomendação	Sev.
P1	Corrigir o fail-open de autenticação (F1): produção nunca deve rodar sem CF_ACCESS_TEAM_DOMAIN/AUD válidos.	CRÍTICA
P1	Implementar de fato a checagem de REFRESH_TOKEN e/ou rate limiting em POST /api/refresh (F3).	ALTA
P2	Decidir e documentar o modelo de autorização (uso interno total-acesso vs. RBAC por conta) e implementar checagem de posse se não for total-acesso deliberado (F2).	ALTA
P2	Neutralizar injeção de fórmula na exportação CSV de negativas (F4).	MÉDIA
P3	Restringir /api/introspect a ambiente de desenvolvimento/staging (F5).	BAIXA
P3	Adicionar teste automatizado que falhe o build/deploy se CF_ACCESS_TEAM_DOMAIN/AUD estiverem vazios em modo produção.	INFORMATIVA

Issues para o GitHub

Texto completo em Markdown, pronto para copiar e colar na criação de cada issue no repositório.

--- ISSUE 1 ---

[Segurança] Autenticação falha aberta (fail-open) quando CF_ACCESS_TEAM_DOMAIN/CF_ACCESS_AUD não estão configurados

```
[Segurança] Autenticação falha aberta (fail-open) quando CF_ACCESS_TEAM_DOMAIN/CF_ACCESS_AUD não estão configurados

**Labels sugeridas:** security, severidade:critica

## Descrição do problema
accessMiddleware() só valida o JWT do Cloudflare Access se as vars CF_ACCESS_TEAM_DOMAIN e CF_ACCESS_AUD estiverem preenchidas. Se qualquer uma estiver vazia – que é exatamente o valor commitado em wrangler.toml (linhas 30-31, CF_ACCESS_TEAM_DOMAIN = "" e CF_ACCESS_AUD = "") – a verificação é pulada inteiramente e o usuário vira { email: "dev@localhost" } com acesso total, sem exigir nenhuma credencial.

## Por que é explorável
Condição de exploração: deploy feito antes do passo 4 do DEPLOY.md (criar a Cloudflare Access Application e rodar `wrangler secret put CF_ACCESS_TEAM_DOMAIN/AUD`), ou qualquer redeploy que rode `wrangler deploy` sem os secrets aplicados (os secrets não são reaplicados automaticamente e o `[vars]` do wrangler.toml sobrescreve com string vazia os nomes homônimos se alguém mover para vars por engano). Nesse cenário a aplicação inteira – dados de custo/receita/conversão de 5 contas de clientes reais, edição de metas e orçamento, criação de anotações e disparo de refresh pago – fica acessível sem login a qualquer pessoa na internet que descubra a URL do Worker.

## Evidência
`worker/auth.ts:88-94 (+ wrangler.toml:29-31)`
...
if (!CF_ACCESS_TEAM_DOMAIN || !CF_ACCESS_AUD) {
  c.set("user", { email: "dev@localhost" });
  return next();
}
...

## Impacto
Exposição total e não autenticada de dados de negócio de clientes da agência (Google Ads + GA4: investimento, receita, conversões, briefing confidencial do cliente) e escrita irrestrita (alterar metas de CPA/orçamento, disparar ETL pago no BigQuery, gerar análises pagas via API da Anthropic).

## Sugestão de correção
Tratar CF_ACCESS_TEAM_DOMAIN/AUD ausentes como erro de configuração em produção, não como bypass: checar `c.env.ENvironment` (ou similar) e só permitir o fallback dev@localhost fora de produção; em produção, retornar 500 (fail-closed) se as vars estiverem vazias. Adicionar validação de startup que rejeite o deploy/serve se essas vars estiverem vazias em ambiente não-dev.

## Critérios de aceite
- [ ] Comportamento corrigido conforme a sugestão de correção acima
- [ ] Cenário de exploração descrito não é mais reproduzível
- [ ] Teste (automatizado ou checklist manual) cobrindo a regressão adicionado
- [ ] Revisão de código por outra pessoa antes do merge
```

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

[Segurança] Ausência total de RBAC — qualquer usuário autenticado tem acesso irrestrito de leitura e escrita a todas as contas de clientes

```
[Segurança] Ausência total de RBAC – qualquer usuário autenticado tem acesso irrestrito de leitura e escrita a todas as contas de clientes

**Labels sugeridas:** security, severidade:alta

## Descrição do problema
Não existe conceito de papel (admin/editor/leitor) nem de dono de conta em nenhuma camada do backend. accessMiddleware() só verifica "é um e-mail permitido pela policy do Cloudflare Access" – a partir daí, o mesmo usuário pode ler dados de TODAS as 5 contas (GET /api/report/*, /api/overview, /api/waste, /api/opportunities, /api/decision-center), editar metas/orçamento/briefing de qualquer conta (POST
```

```

/api/settings/account), criar anotações em nome de qualquer conta (POST /api/annotations), disparar ETL
pago para qualquer conta (POST /api/refresh) e gerar análises de IA pagas para qualquer conta (POST
/api/analysis, POST /api/diagnostico). Não há checagem de posse – é tudo-ou-nada.

## Por que é explorável
Sempre explorável por qualquer conta liberada na policy do Cloudflare Access (hoje: dfassuncao@gmail.com
e/ou @ddlab.com.br, conforme DEPLOY.md). É um risco aceitável se o produto é deliberadamente "ferramenta
interna de agência, todo colaborador vê tudo" – mas isso não está documentado como decisão de produto em
lugar nenhum, e cresce de risco assim que a policy do Access for ampliada (ex.: incluir um cliente final,
um estagiário, um freelancer pontual).

## Evidência
`worker/routes/reports.ts:438-491, worker/index.ts:41-53, worker/routes/analysis.ts:367-442`
...
reports.post("/settings/account", async (c) => {
const body = await c.req.json<{ id: string; ... }>();
if (!body.id) return c.json({ error: "id obrigatório" }, 400);
await c.env.DB.prepare(`UPDATE dim_account SET ... WHERE id = ?`)...
// nenhuma verificação de quem pode editar a conta `body.id`
});
...

## Impacto
Qualquer pessoa com acesso à ferramenta pode alterar configurações financeiras de qualquer cliente,
disparar custos reais (BigQuery, Anthropic) em qualquer conta, e ver o briefing confidencial de todos os
clientes da agência – não apenas dos que gerencia.

## Sugestão de correção
Definir explicitamente o modelo de autorização: se for intencional (equipe interna enxuta), documentar a
decisão; se não, adicionar uma tabela de vínculo usuário↔conta (ou papel) e checar posse em cada endpoint
de escrita antes de aplicar a mutação.

## Critérios de aceite
- [ ] Comportamento corrigido conforme a sugestão de correção acima
- [ ] Cenário de exploração descrito não é mais reproduzível
- [ ] Teste (automatizado ou checklist manual) cobrindo a regressão adicionado
- [ ] Revisão de código por outra pessoa antes do merge
--- FIM ISSUE 2 ---

```

--- ISSUE 3 ---

[Segurança] Rotas administrativas/de debug sem controles adicionais (refresh sem rate limit, introspect exposto)

```

[Segurança] Rotas administrativas/de debug sem controles adicionais

**Labels sugeridas:** security, severidade:alta

---

### Endpoint POST /api/refresh dispara custo real (BigQuery) sem limite de taxa; a proteção REFRESH_TOKEN
documentada nunca é verificada no código (worker/index.ts:41-53, .dev.vars.example:15-16)

**Descrição:** .dev.vars.example documenta REFRESH_TOKEN como "Token simples para proteger POST
/api/refresh quando chamado fora do Access", e Env.REFRESH_TOKEN existe em worker/env.ts:15. Porém nenhum
arquivo do worker lê ou compara REFRESH_TOKEN – o handler `app.post("/api/refresh", ...)` em
worker/index.ts:41 dispara `runEtl()` (que consulta o BigQuery, custo real por bytes processados) sem
checar esse token nem qualquer outro controle além do accessMiddleware() genérico, e sem rate limiting.

**Por que é explorável:** Explorável por qualquer usuário autenticado (ou, combinado com F1, por qualquer
visitante não autenticado): chamadas repetidas a /api/refresh?days=400 para as 5 contas geram consultas
pesadas e repetidas ao BigQuery (a doc do próprio README/DEPLOY recomenda backfill de 400 dias), inflando
a fatura do GCP sem limite.

**Evidência:**
...
app.post("/api/refresh", async (c) => {
const account = c.req.query("account") || undefined;
// ... nenhuma verificação de c.env.REFRESH_TOKEN
c.executionCtx.waitUntil(runEtl(c.env, { accountIds: ..., lookbackDays: ..., facts }));
return c.json({ started: true, ... });
});

```

```

...

**Impacto:** Consumo de custo (billing do GCP/BigQuery) fora de controle, possível esgotamento de cota.

**Sugestão de correção:** Implementar de fato a checagem de REFRESH_TOKEN (ou removê-la da documentação se não for usada) e adicionar rate limiting/debounce por conta (ex.: recusar novo refresh se um já rodou nos últimos N minutos, usando meta_refresh.last_run_at).

---

### GET /api/introspect expõe o schema interno do BigQuery (nomes de coluna/tabela) para qualquer usuário autenticado (worker/index.ts:23-38)

**Descrição:** O endpoint de depuração /api/introspect roda `SELECT column_name, data_type FROM INFORMATION_SCHEMA.COLUMNS` para a tabela informada via querystring e devolve o schema. Está corretamente sob accessMiddleware() e o parâmetro `table` é validado por regex (`^[a-zA-Z0-9_]+$`) antes de entrar na query (sem SQL injection), mas é uma rota de debug interno deixada acessível em produção para qualquer usuário do Access, não só para quem ajusta o ETL.

**Por que é explorável:** Baixo risco – exige já estar autenticado via Access; vaza apenas nomes/tipos de coluna do dataset do BigQuery, não dados de linha.

**Evidência:**
...
app.get("/api/introspect", async (c) => {
  const table = c.req.query("table");
  if (!table || !/^[a-zA-Z0-9_]+$/.test(table)) return c.json({ error: "?table inválido" }, 400);
  ...
});
...

**Impacto:** Reconhecimento de schema interno por um usuário autenticado que não deveria ter esse nível de acesso operacional.

**Sugestão de correção:** Mover para uma rota administrativa separada, ou gate-la atrás de uma flag de ambiente (só disponível em dev/staging).

## Critérios de aceite
- [ ] REFRESH_TOKEN validado em POST /api/refresh (ou removido da documentação se descontinuado)
- [ ] Rate limiting/debounce por conta implementado em POST /api/refresh
- [ ] /api/introspect restrito a ambiente não-produção
- [ ] Teste cobrindo a regressão adicionado

--- FIM ISSUE 3 ---

```

--- ISSUE 4 ---

[Segurança] Exportação CSV de negativas não neutraliza injeção de fórmula (CSV/Excel formula injection)

```

[Segurança] Exportação CSV de negativas não neutraliza injeção de fórmula (CSV/Excel formula injection)

**Labels sugeridas:** security, severidade:media

## Descrição do problema
downloadNegativesCsv() escapa aspas duplas nos campos `label` (termo de busca) e `campaign`, mas não verifica se o valor começa com `=`, `+`, `-` ou `@` – caracteres que o Excel/Google Sheets interpretam como início de fórmula. `label` nesse relatório vem de `search_term` do Google Ads (fact_searchterm_daily), que por sua vez reflete termos de busca reais digitados por usuários – um dado de origem externa que a agência não controla totalmente.

## Por que é explorável
Exige que (a) um termo de busca real acionando os anúncios do cliente contenha uma fórmula válida de planilha logo no início do texto – algo incomum mas não impossível de ser induzido por um agente malicioso pesquisando termos elaborados –, e (b) o operador da DDLab abra o CSV exportado no Excel/Sheets com fórmulas habilitadas. Severidade limitada pela baixa probabilidade de ambas condições, mas o padrão de exportação deveria ser seguro por padrão.

## Evidência
`src/pages/ReportPage.tsx:52-68`
...

```

```
const term = String(r.label ?? "").replace(/"/g, '');
const camp = String(r.campaign ?? "").replace(/"/g, '');
return `${camp}`, "${term}", Phrase, Campaign`;
```

Impacto
Execução de fórmula/DDE ao abrir o CSV no Excel, potencialmente levando a exfiltração de dados locais ou execução de comando (dependendo da configuração do Excel do operador).

Sugestão de correção
Prefixar com um apóstrofo (ou aspas simples neutralizando) qualquer campo que comece com `=`, `+`, `-` ou `@` antes de montar a linha CSV, tanto para `label` quanto `campaign`.

Critérios de aceite
- [] Comportamento corrigido conforme a sugestão de correção acima
- [] Cenário de exploração descrito não é mais reproduzível
- [] Teste (automatizado ou checklist manual) cobrindo a regressão adicionado
- [] Revisão de código por outra pessoa antes do merge

--- FIM ISSUE 4 ---
```